

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

Lenguaje Ensamblador

Lenguaje Ensamblador

- **Definición:** Es un lenguaje traductor (**recibe como entrada código escrito en un cierto lenguaje y produce como salida código en otro lenguaje**) que permite tener un control total sobre cada instrucción generada por una máquina en particular mediante el programa traductor, llamado *Ensamblador*.
- Es un código nemotécnico utilizado para escribir código de máquina mediante expresiones abreviadas.
- Se encuentra muy ligado al Hardware, al contrario que un lenguaje de alto nivel.
- Su ejecución es más rápida.
- Permite trabajar directamente con los registros del microprocesador, la memoria, dispositivos de E/S, etc.

Estructura de un Programa

✓ Básicamente los programas pueden estar englobados en cuatro bloques fundamentales:

- **DECLARACIONES Y DEFINICIONES**
- **SEGMENTO DE DATOS**
- **SEGMENTO DE PILA**
- **SEGMENTO DE CÓDIGO**

DECLARACIONES Y DEFINICIONES

- Donde se realiza la definición de constantes, importación de código o información para el compilador

Ejemplo:

```
CR EQU 13          ; Constante CR (Retorno de carro) en Decimal o bien,  
LF EQU 0Ah         ; Constante LF (Salto de línea) en Hexadecimal
```

SEGMENTO DE DATOS

- Reserva de espacio para las variables y constantes que utiliza el programa.
- Para la creación de un segmento, se utiliza la directiva SEGMENT, la cual indica el comienzo del mismo.
- El final del segmento viene dado por la directiva ENDS (END Segment).

Ejemplo:

```
DATOS SEGMENT      ; Inicio del segmento de datos  
    ;-----  
    MENSAJE DB CR,LF,'Hola Mundo !',CR,LF,'$'  
    ;-----  
DATOS ENDS         ; Fin del segmento de datos
```

SEGMENTO DE PILA

- ✓ Todos los programas han de llevar un pila, con el fin de depositar la información necesaria para las llamadas a funciones, o bien almacenar datos temporalmente. En este bloque se define un tamaño para la pila.
- ✓ El segmento de pila se define como cualquier otro segmento, pero especificando la palabra reservada STACK.
- ✓ En este segmento reservamos únicamente espacio para la pila, es decir, definimos su tamaño.

Ejemplo:

```
PILA SEGMENT STACK                ; Inicio del segmento de pila
    ;-----
    DB 64 DUP('PILA')              ; Inicialización de la pila
    ;-----
PILA ENDS                          ; Fin del segmento de pila
```

Definimos el tamaño de la pila, en este ejemplo añadimos 64 veces los bytes ASCII de la palabra PILA, con lo que otorgamos a la pila un tamaño de 256 bytes (64 x 4 bytes de la palabra PILA = 256 bytes).

SEGMENTO DE CÓDIGO

Definición de todos los procedimientos e instrucciones del programa.

Un procedimiento es un conjunto de sentencias, las cuales se engloban entre la directiva PROC (PROCedure) y la directiva ENDP (END Procedure).

Ejemplo:

```
CODIGO SEGMENT ; Inicio del segmento de código
proc1 PROC FAR ; Inicio procedimiento principal proc1
    ASSUME CS:CODIGO,DS:DATOS,SS:PILA ; Asignar segmentos
    MOV AX,DATOS ; AX=Dirección del segmento de datos
    MOV DS,AX ; DS=AX. Indicar el segmento de datos
    LEA DX,MENSAJE ; DS:DX = dirección de MENSAJE
    MOV AH,9 ; Función DOS para escribir texto en pantalla
    INT 21H ; Llamar a la interrupción del DOS
    MOV AX,4C00H ; Función para terminar el programa
    INT 21H ; Volver al DOS
proc1 ENDP ; Fin del procedimiento proc1
CODIGO ENDS ; Fin del segmento código
END proc1 ; Empezar a ejecutar el procedimiento proc1
```

Los elementos del código pueden ser visualizados mediante cuatro columnas:

ETIQUETA
INICIO

OPERADOR
ADD

OPERANDOS
al,02h

COMENTARIOS
; Suma 02h al contenido de al

ETIQUETAS :

- Permiten caracteres alfanuméricos
- Máximo: 31 caracteres

OPERADOR :

- En esta columna se situarán las instrucciones. **El campo del código de operación es el único que nunca puede estar vacío**, éste siempre contiene una instrucción o una directiva del ensamblador.

OPERANDOS :

- El campo de operandos o de dirección puede contener una dirección o un dato, o puede estar en blanco.

COMENTARIOS (opcional):

- Este campo es opcional. Aquí se situará cualquier comentario personalizado. Estos son útiles para comprender qué hace un programa sin tener que descifrar el código entero. El compilador (ensamblador) ignorará todo texto más allá del carácter punto y coma ";" y hasta el fin de línea.

Juegos de Instrucciones

Las instrucciones del 8086/88 se pueden dividir en varios grupos:

- ✓ Instrucciones de transferencia de datos
- ✓ Instrucciones aritméticas
- ✓ Instrucciones lógicas
- ✓ Instrucciones de desplazamiento y rotaciones
- ✓ Instrucciones de E/S
- ✓ Instrucciones de control del flujo del programa
- ✓ Instrucciones de cadena de caracteres

Instrucciones de transferencia de datos

- ✓ Las instrucciones de transferencia de datos copian datos de un sitio a otro.
- ✓ La instrucción “**MOV**” realiza la transferencia de datos del operando de origen al destino.
- ✓ “**MOV**” admite todos los tipos de direccionamiento.
- ✓ Ambos operandos deben ser del mismo tamaño y **NO** pueden estar ambos en memoria.

Instrucciones aritméticas

Este tipo de instrucciones realizan operaciones aritméticas con los operandos.

Ejemplos:

“**ADD**” y “**ADC**” realizan la suma y la suma con acarreo (bit **CF** del registro de estado) de dos operandos, respectivamente, y guardan el resultado en el primero de ellos. Admiten todos los Modos de Direccionamiento.

“**MUL**” realiza la multiplicación del contenido de **AX** y del operando indicado, guardando el resultado en **AX**, para operaciones de **8 bits** y en **DX:AX** para operaciones de **16 bits**.

Instrucciones lógicas

Realizan las operaciones lógicas: **OR, XOR, AND, NOT, TEST, CMP**.

OR, XOR y AND realizan las operaciones lógicas “O”, “O exclusiva” e “Y”, respectivamente, de dos operandos, guardando el resultado en el primero de ellos. Estas operaciones se efectúan **bit a bit**. Admiten todos los Modos de Direccionamiento.

NOT realiza la operación de negado lógico de los bits del Operando (**CA₁**), guardando el resultado en el mismo operando.

Admite todos los Modos Direccionamiento, excepto el Modo Inmediato.

Instrucciones de Comparación

Estas instrucciones realizan funciones de comparación no guardando el resultado, pero si afecta al registro de estado (**no alteran los operandos**). Son muy útiles en las instrucciones de salto que se verán más adelante.

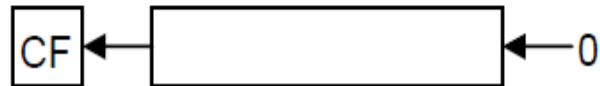
TEST realiza la operación lógica “Y” de dos operandos, pero **NO** afecta a ninguno de ellos, **SÓLO** afecta al registro de estado.

CMP realiza la resta de los dos operandos (como la instrucción **SUB**) pero **NO** afecta a ninguno de ellos, **SÓLO** afecta al registro de estado.

Instrucciones de Desplazamiento y Rotaciones

Realizan operaciones de desplazamiento y rotaciones de bits, y son: **SAL/SHL**, **SAR**, **SHR**, **ROL**, **ROR**, **RCL**, **RCR**.

SAL/SHL realiza desplazamiento a la izquierda del primer operando tantos bits como indique el segundo operando, introduciendo un 0 y guardando el bit que sale en el bit CF del registro de estado.



SHL < D > , < c >

Corrimiento lógico de " c " bits a la izquierda

SHR realiza el desplazamiento a la derecha del operando, introduciendo un 0 y guardando el resultado en el bit CF del registro de estado.

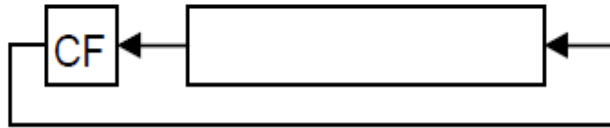


SHR < D > , < c >

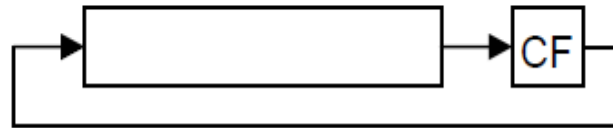
Corrimiento lógico de " c " bits a la derecha

Instrucciones de desplazamiento y rotaciones

RCL realiza la rotación a la izquierda de los bits del operando a través del bit CF (acarreo) del registro de estado.



RCR realiza la rotación a la derecha de los bits de operando a través del bit CF del registro de estado.



ROL realiza la rotación a la izquierda de los bits del operando, ignorando el bit CF del registro de estado, aunque en CF se almacena el bit que se rota.



Instrucciones de E/S

Se usan para la comunicación con los dispositivos periféricos. Y son **IN** y **OUT**.

IN lee de un puerto (sólo si la dirección del puerto es menor que 255).

OUT escribe en un puerto (sólo si la dirección del puerto es menor que 255).

Instrucciones de control del programa

Se utilizan para el control del programa, son instrucciones de salto, bucles y llamadas a procedimientos.

Las instrucciones de un programa se ejecutan unas tras otras, siguen una lógica secuencial, salvo que haya una instrucción de este tipo.

Instrucciones de salto

Estas instrucciones permiten saltar a otras partes del código.

Todas cambian el registro **IP** (contador de programa) y el registro **CS** (segmento de código) si es un salto lejano.

Un salto es lejano cuando la dirección a la que se salta no está en el mismo segmento de código.

Saltos Condicionales e Incondicionales

La mayoría de los programas constan de varios ciclos en los que una serie de pasos se repite hasta alcanzar un requisito específico y varias pruebas para determinar qué acción se realiza de entre varias posibles.

Requisitos como este implican la transferencia de control a la dirección de una instrucción que no sigue de inmediato de la que se está ejecutando actualmente. Una transferencia de control puede ser hacia adelante, para ejecutar una serie de pasos nuevos, o hacia atrás, para volver a ejecutar los mismos pasos.

Ciertas instrucciones pueden transferir el control fuera del flujo secuencial normal añadiendo un valor de desplazamiento al **IP**.

Dirección: Corta, Cercana y Lejana

Una operación de salto alcanza una **dirección corta** por medio de un **desplazamiento de un byte**, limitado a una distancia de **-128 a 127 bytes**.

Una operación de salto alcanza una **dirección cercana** por medio de un **desplazamiento de una palabra**, limitado a una distancia de **-32.768 a 32.767 bytes** **dentro del mismo segmento**.

Una **dirección lejana** puede estar en otro segmento y es alcanzada por medio de una **dirección de segmento y un desplazamiento**.

Etiquetas de Instrucciones

Las instrucciones **JMP**, **Jnnn** (salto condicional) y **LOOP** requieren un operando que se refiere a la etiqueta de una instrucción. El ejemplo siguiente salta a **A90**, que es una etiqueta dada a una instrucción **MOV**:

JMP A90

```
...  
A90: MOV AH, 00  
...
```

La etiqueta de una instrucción, tal como **A90:**, terminada con dos puntos (:) para darle atributo de cercana, esto es, la etiqueta esta dentro de un procedimiento en el mismo segmento de código.

Cuidado: Un error común es la omisión de los dos puntos. Note que una etiqueta de dirección en un operando de instrucción (como **JMP A90**) no posee el carácter de dos puntos.

La instrucción **JMP** (Salto incondicional)

Una instrucción usada comúnmente para la transferencia de control es la instrucción **JMP** (**J**ump, salto, bifurcación).

El salto es **incondicional**, ya que la operación transfiere el control bajo cualquier circunstancia. También **JMP** vacía el resultado de la instrucción previamente procesada; por lo que, un programa con muchas operaciones de salto puede perder velocidad de procesamiento.

El formato general para **JMP** es:

| [etiqueta] | **JMP** | dirección corta, cercana o lejana |

INSTRUCCIONES DE SALTO CONDICIONAL

El ensamblador permite usar una variedad de instrucciones de salto condicional que transfieren el control dependiendo de las configuraciones en el registro de banderas. Por ejemplo, puede comparar dos campos y después saltar de acuerdo con los valores de las banderas que la comparación establece.

El formato general para el salto condicional es:

| [etiqueta:] | Jnnn | dirección corta |

La instrucción LOOP

La instrucción **LOOP**, requiere un valor inicial en el registro CX.

En cada iteración, LOOP de forma automática disminuye 1 de CX. Si el valor en el CX es cero, el control pasa a la instrucción que sigue; si el valor en el CX no es cero, el control pasa a la dirección del operando. La distancia debe ser un **salto corto**, desde -128 hasta +127 bytes. Para una operación que exceda este límite, el ensamblador envía un mensaje como "salto relativo fuera de rango". El formato general de la instrucción LOOP es:

| [etiqueta:] | LOOP | dirección corta |

Ejemplo: “Si... entonces... sino”

Pseudocódigo:

```
SI  AH = BL
    AL = 01h
SINO
    AL = 00h
FIN SI
```

Assembler:

```
MOV AH,[0204h] ; Cargamos AH con el contenido de la dirección 0204h
MOV BL, [0A20h] ; Cargamos BL con el contenido de la dirección 0A20h
CMP AH,BL      ; Compara los contenidos de AH y BL
JNE ET01       ; Salta a “ET01” si los valores comparados son distintos
MOV AL,01h     ; Cargamos AL con el valor 01h si son iguales
JMP ET02       ; Salta a “ET02” para evitar la instrucción siguiente
ET01: MOV AL,00h ; Cargamos AL con el valor 00h si son distintos
ET02: .....    ; Continúa la ejecución del programa
```

Tipos de instrucciones del i8086)

Tipo de instrucción	Efecto
Instrucciones de transferencia	Mueven información entre registros, registros y posiciones de memoria, o entre registros y puertos de entrada/salida
Instrucciones aritméticas	Realizan operaciones aritméticas: sumas, restas, etc.
Instrucciones de manejo de bits	Realizan operaciones de desplazamiento, rotación y lógicas sobre registros o posiciones de memoria
Instrucciones de transferencia de control	Sirven para controlar la ejecución de las instrucciones del programa
Instrucciones de entrada salida	Mueven información entre registros y puertos de entrada/salida
Instrucciones de manejo de cadenas	Realizan operaciones sobre cadena de bytes o palabras
Instrucciones de interrupción	Provocan que el microprocesador realice un servicio que se le solicita diferente a las instrucciones que está ejecutando

Instrucciones de transferencia de datos (I)

- **Nombre:** MOV
- **Formato:** MOV destino, origen
- **Descripción:**
Transfiere un byte o una palabra desde el operando origen al operando destino
- **Ejemplos:**
 - MOV CX, 112h ; CX = 112h
 - MOV BL, 255 ; BL = 255 = FFh
 - MOV ES, AX ; ES = AX
 - MOV AL, 12h ; AL = 12h
 - MOV PAL_MEM, BX ; PAL_MEM = BX

Instrucciones de transferencia de datos (II)

- **Nombre:** PUSH
- **Formato:** PUSH origen
- **Descripción:**
Decrementa el puntero de pila (SP) en 2 y luego transfiere la palabra que se ha especificado en el operando origen a lo alto de la pila
- **Ejemplo:**
 - PUSH BX ; Pone BX en la pila

Instrucciones de transferencia de datos (y III)

- **Nombre:** POP
- **Formato:** POP destino
- **Descripción:**
Transfiere un byte o una palabra desde la cima de la pila al operando destino y luego incrementa la pila en 2
- **Ejemplo:**
 - POP BX ; Pone en BX el contenido de la cima de la pila